

Gaussian Mixture Conditional Variational Recurrent Neural Network for Unified Trajectory Imputation and Prediction

Yi Xu¹, Graduate Student Member, IEEE, and Yun Fu², Fellow, IEEE

Abstract—Predicting trajectories is essential for interpreting human behavior, yet it remains a challenging task when relying solely on observed motion patterns. Despite substantial progress, most existing methods assume fully observed trajectories and fail to account for missing data caused by occlusion, limited field of view, or sensor failures. This limitation substantially compromises the reliability of trajectory prediction, particularly in real-world deployment where observations are often incomplete. In light of this issue, our work presents the Gaussian Mixture Conditional Variational Recurrent Neural Network (GMC-VRNN), which unifies trajectory imputation and prediction within a single framework. Our GMC-VRNN framework couples a Multi-Space Graph Neural Network (MS-GNN) with a Gaussian Mixture Conditional VRNN, further augmented by a Bidirectional Temporal Decay (BTD) module, to achieve robust spatio-temporal representation learning under incomplete observations. To verify its effectiveness, we conduct extensive evaluations on two sports datasets covering multiple scenarios, jointly tackling trajectory imputation and prediction. Our experiments confirm that GMC-VRNN surpasses recent state-of-the-art approaches, offering enhanced precision and stronger robustness under diverse conditions.

Index Terms—Trajectory imputation, trajectory prediction, gaussian mixture model, graph neural network, variational recurrent neural network.

I. INTRODUCTION

TRAJECTORY prediction focuses on forecasting the future positions of agents from observations of their past motion. A wide range of real-world domains relies on this task, such as autonomous driving [1], [2], [3], [4], motion capture [5], [6], and behavior analysis [7], [8], serving as the bridge from understanding past activities to anticipating future intentions. A major difficulty, however, arises from the need to jointly account for multiple determinants of agent motion, including self-dynamics (ego-motion), environmental context, and the complex social interactions among neighboring agents. Among these, social interactions are particularly difficult to quantify due to their complex and subtle nature, prompting significant research efforts to model them effectively.

Received 4 February 2025; revised 20 September 2025; accepted 23 October 2025. Date of publication 31 October 2025; date of current version 4 February 2026. Recommended for acceptance by H. Ling. (Corresponding author: Yi Xu.)

Yi Xu is with the Department of ECE, Northeastern University, Boston, MA 02115 USA (e-mail: xu.yi@northeastern.edu).

Yun Fu is with the Department of ECE and Khoury College of Computer Science, Northeastern University, Boston, MA 02115 USA.

Digital Object Identifier 10.1109/TPAMI.2025.3627470

Traditional approaches, such as the Social Force model [9], design hand-crafted rules to represent behaviors such as leader-following and collision avoidance. More recently, neural network-based methods have been widely adopted to model inter-agent interactions. Notably, the pioneering Social-LSTM [10] employs a pooling operation to aggregate contextual cues across agents, while utilizing a Long Short-Term Memory (LSTM) backbone for trajectory predictions. Building on this idea, numerous methods [11], [12], [13], [14], [15] incorporate strategies like attention modules and similarity-driven modeling, aiming to improve the effectiveness of inter-agent communication. In addition to producing a single deterministic prediction, generative approaches have been explored, including adversarial learning frameworks (GANs) [16], [17], [18], [19], [20] and encoder-decoder schemes [21], [22], [23], [24], which are capable of producing diverse sets of feasible trajectories. The availability of diverse trajectory samples can thus be exploited to uncover possible movement tendencies and behavioral patterns of agents.

Despite significant research efforts and various proposed solutions, many trajectory prediction methods assume complete agent observations, an often impractical assumption. For example, in sports matches such as basketball and football, some players may become occluded due to camera view constraints. Moreover, live cameras typically track and follow the ball, resulting in players becoming temporarily occluded or reappearing depending on their positional relationship with the ball. Fig. 1 illustrates a representative case. At $T = 1$, the ball, indicated by a red circle, is held by a player, and most players from both the offensive and defensive teams are captured within the sports broadcast camera's field of view. At $T = 2$, as the ball flies across the field, the broadcast camera zooms in and tracks the ball, leaving some players outside the frame. By $T = 3$, the ball is caught by another player (also marked with a yellow circle), and only a small subset of players remains visible in the frame. Forecasting trajectories from incomplete observations is inherently difficult, as the missing information compromises both the accuracy of predictions and the ability to reason about agents' behavioral patterns.

Another research area focuses on the sequence imputation problem, with various recent works [25], [26], [27], [28], [29], [30] addressing the challenge. Many prior methods depend on autoregressive modeling of past time steps to recover missing values, a strategy that is susceptible to error propagation and

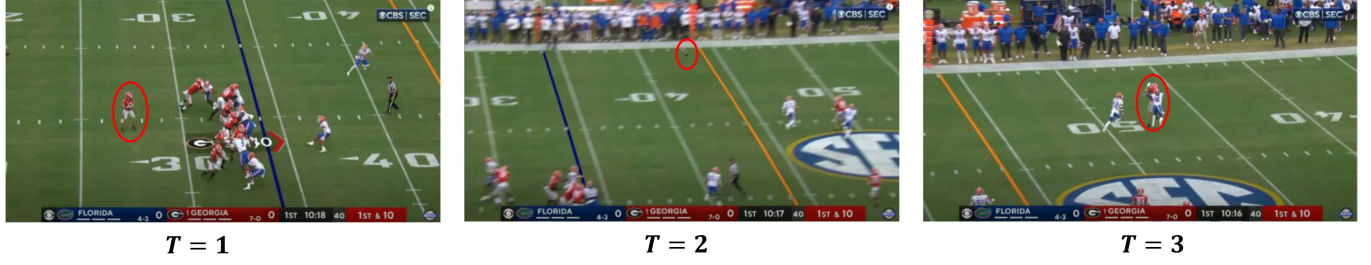


Fig. 1. Illustration of a ball-passing sequence in a football game. The ball, highlighted with a red circle in three consecutive frames, remains the focus of the camera, while the set of visible players progressively diminishes as the view zooms in.

becomes especially unstable in long-term sequence modeling. Furthermore, commonly used benchmarks [31], [32], [33], [34] often lack interacting entities, even though dense interactions are a defining characteristic of multi-agent sports scenarios. Some recent studies [35], [36] have extended imputation methods to multi-agent contexts, achieving promising results. Despite their progress, existing methods often fail to explicitly capture the intrinsic link between imputing missing trajectories and forecasting future ones. In fact, complete trajectory observations are vital for accurate predictions, while precise trajectory prediction can, in turn, reveal temporal correlations between past and future movements. Leveraging this mutual relationship between imputation and prediction can significantly enhance the effectiveness of both tasks.

We present a holistic framework, the Gaussian Mixture Conditional Variational Recurrent Neural Network (GMC-VRNN), aimed at unifying trajectory imputation and prediction, with sports analytics as the primary application domain. A key component of this framework is the Multi-Space Graph Neural Network (MS-GNN), proposed to derive discriminative spatial features even when the input observations are incomplete. Unlike other graph-based methods, our MS-GNN employs three different Graph Convolutional Layers (GCLs) to capture different spatial features, effectively representing and emphasizing spatial missing patterns in the data. Temporal dependencies are captured through a Conditional Variational Recurrent Neural Network (C-VRNN) augmented by a Bidirectional Temporal Decay (BTD) mechanism, specifically tailored to accommodate the irregularities of incomplete observations. Additionally, we replace deterministic regression with a probabilistic formulation, where trajectory outputs are characterized by a bivariate Gaussian Mixture Model (GMM). The core idea behind our approach is to capture spatial-temporal features of missing patterns and unify both objectives through shared parameters, enabling mutual support between the tasks for improved performance. To thoroughly evaluate the tasks of trajectory imputation and prediction in multi-agent settings, we establish benchmarks on two sports datasets, Basketball-TIP and Football-TIP, constructed with realistically generated incomplete trajectories. The primary contributions of this study are summarized as follows:

- We address the joint tasks of trajectory imputation and prediction by developing GMC-VRNN, a unified framework that imputes missing observations and predicts future trajectories simultaneously.
- Our design integrates an MS-GNN to capture spatial dependencies from incomplete observations, together with

a Conditional VRNN augmented by a BTD module to capture temporal dynamics under missing data, facilitating information propagation over time.

- We establish benchmarks on two sports datasets covering diverse scenarios, providing a testbed and strong baselines for evaluating multi-agent trajectory imputation and prediction.
- Comprehensive experiments demonstrate the robustness and effectiveness of our proposed method, namely GMC-VRNN, showing consistent gains over state-of-the-art baselines across multiple scenarios.

The preliminary conference version, GC-VRNN [37], mainly focused on establishing baselines for unified trajectory imputation and prediction. The present paper advances this framework into GMC-VRNN with the following technical improvements and highlights:

- The GC-VRNN [37] method employs a bivariate Gaussian distribution for trajectory modeling, overlooking the multi-modal nature of human motion. GMC-VRNN instead adopts a GMM formulation, which captures diverse trajectory patterns, generates multiple plausible modalities per agent, and enhances prediction diversity. Furthermore, the temporal decay mechanism is upgraded to a bidirectional variant, allowing more effective modeling of temporal missing patterns.
- While the earlier work [37] introduced two sports datasets and one vehicle dataset with various masking strategies, the present study concentrates specifically on sports domains. We further enrich the two sports datasets by extending them to cover ten distinct scenarios each, thereby providing a more comprehensive evaluation benchmark.
- Through extensive experiments and detailed ablation analyses, GMC-VRNN is shown to consistently outperform GC-VRNN, yielding substantial performance gains and validating the effectiveness of our proposed enhancements.

The rest of the paper is organized as follows. Section II reviews related work on trajectory imputation and prediction. Section III describes the architecture and methodological components of GMC-VRNN. Section IV reports the experimental setup and results, and Section V concludes the paper.

II. RELATED WORK

This section offers an overview of existing research in two related areas: trajectory prediction and trajectory imputation.

A. Trajectory Prediction

The goal of trajectory prediction is to estimate where agents will move next, given their historical observations. A critical factor to consider is the dense interactions among agents, which are dynamic and subtle, making them challenging to quantify in real-world applications. One of the earliest deep learning frameworks for trajectory prediction, Social-LSTM [10], introduces a pooling layer to aggregate inter-agent context, enabling interaction patterns to be learned directly from data. Building on this foundation, many following methods [12], [13], [23], [38] have been proposed to apply attention mechanisms and graph-based models, to extract representative interaction features. Bridging trajectory prediction with downstream tasks such as planning and decision-making requires accounting for the inherent multi-modality of human motion, since agents in the real world can pursue multiple plausible future trajectories. A range of generative frameworks, including adversarial models [17], [18], [19], [39], [40], variational autoencoders [21], [22], [41], [42], [43], and diffusion-based approaches [44], [45], [46], have been employed to address this issue. These models are capable of generating diverse, high-fidelity trajectory samples that align closely with the complexity of real-world scenarios. Our previous work [37] points out that existing trajectory prediction methods often assume complete observations, a condition rarely met in real-world scenarios. This limitation is particularly evident in sports such as basketball or football, where missing observations are common due to occlusion or restricted camera views. In response to this challenge, we present a unified framework designed to simultaneously perform trajectory imputation and prediction, thereby offering a holistic solution.

B. Trajectory Imputation

As a classic research topic, imputation has been extensively studied over the years. One of the simplest techniques involves substituting missing values with the mean or median [47], which performs poorly in complex scenarios. Alternative methods, such as linear fitting [48], k-nearest neighbors (KNN) [49], [50], and the expectation-maximization (EM) algorithm [51], [52], have also been explored. However, these approaches rely on rigid priors, limiting their ability to generalize effectively. More flexible frameworks leverage generative methods to learn missing patterns. For example, deep autoregressive models based on RNNs [53], [54], [55], [56] have been proposed to impute sequential data, while other methods [25], [27], [57], [58], [59], [60] focus on generating reconstructed sequences. Despite recent progress in imputation, few works address trajectory imputation in multi-agent scenarios. For example, NAOMI [35] develops a non-autoregressive method that leverages the multi-scale structure inherent in sequential data. Likewise, GMAT [36] adopts a hierarchical architecture that generates approximate macro-intent annotations to facilitate sequence generation. However, these approaches solely address trajectory imputation while neglecting the prediction task. INAM [57] addresses both imputation and prediction using an imitation learning framework but operates in an asynchronous mode. Additionally, INAM is evaluated on a single dataset where

missing instances are randomly generated, making the masking strategy less realistic for real-world applications. In our previous work [37], we proposed an end-to-end learning paradigm that jointly addresses trajectory imputation and prediction within a unified model. In this extended work, we continue to tackle both tasks simultaneously, enhancing the model design with a bidirectional temporal decay module. This improvement enables better performance compared to the unidirectional approach.

III. METHODOLOGY

This section begins by formalizing the unified problem of trajectory imputation and prediction. Subsequently, we elaborate on the GMC-VRNN framework, which is composed of two core modules: (1) the Multi-Space Graph Neural Network (MS-GNN) for capturing spatial relations, and (2) the Conditional Variational Recurrent Neural Network (C-VRNN) enhanced with a Bidirectional Temporal Decay (BTD) mechanism for temporal modeling. Fig. 2 provides a schematic overview, illustrating how information is propagated through the full trajectory within the proposed design.

A. Problem Definition

Consider a group of N agents denoted as $\Omega = 1, 2, \dots, N$, observed from time t_1 to t_p . The trajectory of agent i up to t_p is given by $X_i^{\leq t_p} = \{\mathbf{x}_i^1, \dots, \mathbf{x}_i^t, \dots, \mathbf{x}_i^{t_p}\}$, where each $\mathbf{x}_i^t \in \mathbb{R}^2$ specifies the 2D coordinates of the agent at time t . Collectively, the trajectories of all agents form $X_\Omega^{\leq t_p} = \{X_i^{\leq t_p} | \forall i \in \Omega\}$. Since the trajectories are often incomplete, we introduce a binary masking matrix $M_i^{\leq t_p} = \{\mathbf{m}_i^1, \dots, \mathbf{m}_i^t, \dots, \mathbf{m}_i^{t_p}\}$, where $\mathbf{m}_i^t = 1$ denotes that the trajectory of agent i is observed at time t , and $\mathbf{m}_i^t = 0$ indicates it is missing.

The unified task of trajectory imputation and prediction can therefore be described by two objectives: (1) imputing the missing observations of agents within the interval $[t_1, t_p]$, and (2) generating multiple K plausible future trajectories from t_{p+1} to t_f , conditioned on incomplete histories. Formally, the problem is cast as learning a model $f(\cdot)$ parameterized by W^* that outputs both the imputed trajectories $\hat{X}_\Omega^{\leq t_p}$ and the predicted set of diverse futures $\hat{Y}_\Omega^{t_{p+1} \leq t \leq t_f}$.

B. Multi-Space Graph Neural Network

Leveraging graph structures is an intuitive way to represent spatial interactions among multiple agents. Nevertheless, handling incomplete observations requires a more customized design. To this end, we introduce the Multi-Space Graph Neural Network (MS-GNN), a framework that augments Graph Convolutional Layers (GCL) to robustly capture spatial features when agent trajectories are partially missing.

1) *Graph Definition*: We formalize the spatial graph construction as follows. At each time step t , each agent is represented as a node, and the graph is defined as $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t)$, where $\mathcal{V}^t = \{v_i^t | i \in \Omega\}$ denotes the set of nodes (agents), and $\mathcal{E}^t = \{e_{i,j}^t | i, j \in \Omega\}$ specifies the edge set. The connectivity is encoded by an adjacency matrix $\mathbf{A}^t = \{a_{i,j}^t | i, j \in \Omega\}$. The corresponding node features are defined as $\mathbf{F}^t = \{\mathbf{f}_i^t \in \mathbb{R}^D | i \in \Omega\}$.

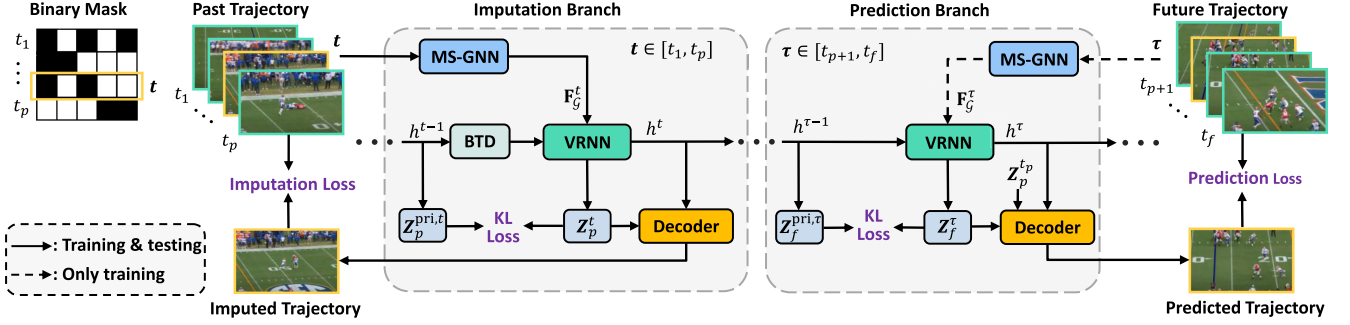


Fig. 2. Overview of the GMC-VRNN framework. The inputs include past incomplete trajectories, their associated masks, and future trajectories, and the outputs are the imputed and predicted trajectories. The framework unifies trajectory imputation and prediction within an end-to-end architecture.

$\Omega\}$, where $\mathbf{f}_i^t$ indicates the D -dimensional feature vector of agent i at time t .

2) *Graph Input*: The MS-GNN takes three components as input: partially observed trajectories, a binary mask indicating missing entries, and future trajectories (available only during training). For agent i at time step t , the corresponding node feature is obtained by mapping these inputs into a high-dimensional embedding space, which is expressed as:

$$\mathbf{f}_i^t = \begin{cases} \varphi_p((\mathbf{x}_i^t \odot \mathbf{m}_i^t) \oplus \mathbf{m}_i^t; \mathbf{W}_p) & t \in [t_1, t_p] \\ \varphi_f(\mathbf{y}_i^t; \mathbf{W}_f) & t \in [t_{p+1}, t_f] \end{cases}, \quad (1)$$

where $\varphi_p(\cdot)$ and $\varphi_f(\cdot)$ represent distinct projection mappings with associated weight matrices $\mathbf{W}_p \in \mathbb{R}^{3 \times D}$ and $\mathbf{W}_f \in \mathbb{R}^{2 \times D}$. These projections are implemented using MLPs in our framework. Here, $\odot$ denotes element-wise multiplication and $\oplus$ the concatenation operation. The variable $\mathbf{y}_i^t$ represents the future position of agent i , which is employed exclusively during training to provide supervision.

The MS-GNN employs three separate graph convolutional layers (GCLs) at each time step, each operating on a distinct feature space to extract spatial representations. These layers are designed not only to capture essential features but also to emphasize spatial correlations induced by missing data patterns. For clarity in notation, we drop the superscript t hereafter, writing $\mathbf{f}_i^t$ as $\mathbf{f}_i$ and $\mathbf{F}^t$ as $\mathbf{F}$.

3) *Static Topology GCL*: In the standard GCL [61], the adjacency matrix encodes only the pairwise connectivity between nodes, where $\mathbf{A}_{i,j} = 1$ if there exists an edge from node i to j , and $\mathbf{A}_{i,j} = 0$ otherwise. In contrast, our setting involves partially observed trajectories, where some agents (nodes) may be absent at certain time steps. To accommodate this, we introduce a modified adjacency matrix $\mathbf{A}_{st}$ in the static topology GCL, which jointly encodes both connectivity and node visibility. Correspondingly, the identity matrix $\mathbf{I}_{st}$ is redefined with an additional constraint to incorporate self-loops. The formulation of these constraints is given as follows:

Constraint 1: At time step t , $\mathbf{A}_{i,j}^t = \mathbf{A}_{j,i}^t = 1$ when both nodes i and j are visible; otherwise, $\mathbf{A}_{i,j}^t = \mathbf{A}_{j,i}^t = 0$.

Constraint 2: For the identity matrix $\mathbf{I}^t$, $\mathbf{I}_{i,i}^t = 1$ if node i is visible at time step t , and $\mathbf{I}_{i,i}^t = 0$ otherwise.

Following [61], the propagation rule for the l -th static topology layer is formulated as:

$$\mathbf{F}_{st}^{(l+1)} = \sigma(\hat{\mathbf{A}}_{st} \mathbf{F}_{st}^{(l)} \mathbf{W}_{st}^{(l)}), \quad (2)$$

with the normalized adjacency matrix defined as $\hat{\mathbf{A}}_{st} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A}_{st} + \mathbf{I}_{st})\tilde{\mathbf{D}}^{1/2}$. Here, $\tilde{\mathbf{D}}$ is the diagonal degree matrix of $\mathbf{A}_{st} + \mathbf{I}_{st}$, $\mathbf{W}_{st}^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l+1)}}$ represents the learnable parameters of the l -th layer, and $\sigma(\cdot)$ corresponds to the ReLU activation. This static GCL provides a fixed mechanism for encoding both connectivity and visibility among agents.

4) *Dynamic Learnable GCL*: Unlike $\mathbf{A}_{st}$, which consists of binary and static connections, the dynamic learnable variant adopts an unconstrained adjacency matrix $\mathbf{A}_{dl}$, inspired by [62]. This matrix adaptively models the strength of inter-node relationships, thereby improving the flexibility of the GCL. Initialized with random values, $\mathbf{A}_{dl}$ is optimized during training to adjust the connectivity by reinforcing, attenuating, adding, or removing edges as required. Following the formulation in (2), the propagation of the graph feature $\mathbf{F}_{dl}^{(l)}$ through the l -th dynamic learnable layer can be expressed as:

$$\mathbf{F}_{dl}^{(l+1)} = \sigma(\mathbf{A}_{dl} \mathbf{F}_{dl}^{(l)} \mathbf{W}_{dl}^{(l)}), \quad (3)$$

where $\mathbf{W}_{dl}^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l+1)}}$ represents the learnable weight matrix of the l -th dynamic learnable layer. Because every entry in $\mathbf{A}_{dl}$ is unconstrained and updated during training, the matrix is generally asymmetric, enabling each directed edge to adaptively learn its own contribution to the feature update of connected nodes. Intuitively, this dynamic formulation offers greater flexibility than the static topology GCL, as it can more effectively capture heterogeneous relationships among agents.

5) *Edge Conditioned GCL*: The two variants described above capture spatial relationships among nodes using different formulations of the adjacency matrix. However, in scenarios with incomplete observations, a key challenge arises due to missing node features for certain agents. To address this issue and explicitly capture spatial missing patterns, we introduce an edge-conditioned GCL. In this design, each edge is assigned a categorical label, and the associated information is integrated into the graph propagation process.

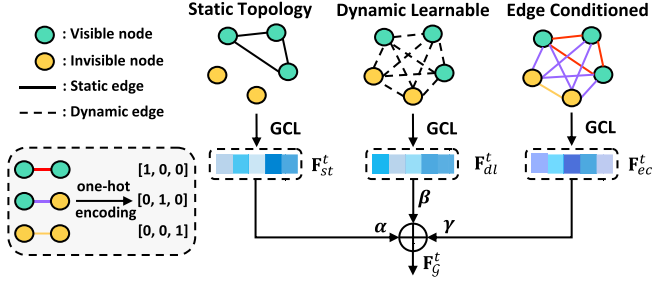


Fig. 3. Schematic depiction of three distinct graph convolutional layers applied to agents at time step t . The resulting feature representations are combined into $\mathbf{F}_G^t$ via a weighted integration.

As illustrated in Fig. 3, the edge-conditioned GCL distinguishes three categories of edges based on the visibility of node pairs. These categories are represented as one-hot vectors $\vartheta_{i,j} \in \mathbb{R}^3$, which are processed by a mapping network $\varphi_{ec}(\cdot)$ to produce edge-specific transformation matrices $\Theta_{i,j} \in \mathbb{R}^{D_G \times D}$ or node feature updating. The update rule is defined as:

$$\begin{aligned} \mathbf{f}_{ec;i} &= \frac{1}{|\mathcal{V}(i)|} \sum_{j \in \mathcal{V}(i)} \varphi_{ec}(\vartheta_{i,j}; \mathbf{W}_{ec}) \mathbf{f}_j + \mathbf{b}_{ec} \\ &= \frac{1}{|\mathcal{V}(i)|} \sum_{j \in \mathcal{V}(i)} \Theta_{i,j} \mathbf{f}_j + \mathbf{b}_{ec}, \end{aligned} \quad (4)$$

where $\mathbf{f}_{ec;i}$ denotes the feature vector of node i in $\mathbf{F}_{ec}$, $\mathbf{W}_{ec}$ and $\mathbf{b}_{ec}$ are the learnable parameters and bias of the edge-conditioned layer, and $\mathcal{V}(i)$ specifies the neighborhood of node i . In practice, $\varphi_{ec}(\cdot)$ is implemented as a Conv2D block consisting of two Conv2D layers followed by an average pooling layer. This edge-conditioned GCL is only applied to the observed trajectories.

6) *Graph Feature Fusion*: The feature representations obtained from the final layers of the three GCL variants, $\mathbf{F}_{st}$, $\mathbf{F}_{dl}$, and $\mathbf{F}_{ec}$, are integrated into a unified representation $\mathbf{F}_G \in \mathbb{R}^{N \times D_G}$. The fusion is formulated as:

$$\mathbf{F}_G = \alpha \mathbf{F}_{st} + \beta \mathbf{F}_{dl} + \gamma \mathbf{F}_{ec}, \quad (5)$$

where the output dimension of each GCL is set to D_G , and α , β , and γ denote learnable fusion parameters, each of dimension $\mathbb{R}^{D_G}$. This formulation enables the model to adaptively balance contributions from static, dynamic, and edge-conditioned relational features.

C. Conditional Variational Recurrent Neural Network With Bidirectional Temporal Decay

To capture temporal dependencies across both observed and future trajectories, we adopt a Conditional Variational Recurrent Neural Network (C-VRNN). Within the imputation branch, we further incorporate a Bidirectional Temporal Decay (BTD) module, which explicitly models the elapsed time between consecutive observations and the current step. Additionally, the imputation and prediction branches share parameters through a recurrent temporal flow, ensuring that trajectory priors and latent variables are jointly updated. This design implicitly promotes information exchange between the two branches.

The original VRNN [63] can be regarded as a variant of the Variational Autoencoder (VAE) conditioned on the hidden states of an RNN. It is trained by maximizing the sequential Evidence Lower Bound (ELBO):

$$\mathbb{E}_{q_\phi(\mathbf{z}^{\leq T} | \mathbf{x}^{\leq T})} \left[\sum_{t=1}^T \log p_\theta(\mathbf{x}^t | \mathbf{z}^{\leq t}, \mathbf{x}^{< t}) - \text{KL}(q_\phi(\mathbf{z}^t | \mathbf{x}^{\leq t}, \mathbf{z}^{< t}) || p_\theta(\mathbf{z}^t | \mathbf{x}^{< t}, \mathbf{z}^{< t})) \right]. \quad (6)$$

1) *Prior*: The prior distribution of the latent variable $\mathbf{z}^t$ at time step t is modeled as:

$$\mathbf{z}^t \sim \mathcal{N}(\boldsymbol{\mu}^{\text{pri},t}, \boldsymbol{\sigma}^{\text{pri},t^2}), \quad (7)$$

where the mean $\boldsymbol{\mu}^{\text{pri},t}$ and variance $\boldsymbol{\sigma}^{\text{pri},t^2}$ are parameterized by the RNN hidden state h^{t-1} as:

$$[\boldsymbol{\mu}^{\text{pri},t}, \boldsymbol{\sigma}^{\text{pri},t^2}] = \varphi^{\text{pri}}(h^{t-1}; \mathbf{W}^{\text{pri}}), \quad (8)$$

where $\varphi^{\text{pri}}(\cdot)$ denotes a mapping network that converts the hidden representation into the parameters of the prior distribution, with learnable weights $\mathbf{W}^{\text{pri}}$.

2) *Generation*: At each time step t , the generative process decodes either the imputed trajectory or the predicted future positions from the latent variables. In contrast to our earlier work [37], which employed a bivariate Gaussian distribution, we here adopt a bivariate Gaussian Mixture Model (GMM) to capture multi-modality in trajectory prediction, formulated as:

$$\begin{aligned} \mathbb{P}(\mathbf{x}^{t_1:t_p}) &= \sum_{k=1}^K p_k \prod_{t=t_1}^{t_p} \mathcal{N}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t, \boldsymbol{\rho}^t) \\ \mathbb{P}(\mathbf{y}^{t_{p+1}:t_f}) &= \sum_{k=1}^K p_k \prod_{t=t_{p+1}}^{t_f} \mathcal{N}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t, \boldsymbol{\rho}^t), \end{aligned} \quad (9)$$

where K represents the number of modalities for each trajectory, and p_k denotes the mixing weight of each bivariate Gaussian distribution component. Within each Gaussian mixture component, $\boldsymbol{\mu}^t$ is the mean, $\boldsymbol{\sigma}^t$ the standard deviation, and $\boldsymbol{\rho}^t$ the correlation coefficient.

In the imputation branch, the generation process is conditioned on the latent variable $\mathbf{z}^t$ and the previous hidden state h^{t-1} as:

$$\hat{\mathbf{x}}^t \sim \varphi_p^{\text{dec}}(\varphi_p^z(\mathbf{z}^t) \oplus h^{t-1}; \mathbf{W}_p^{\text{dec}}) \quad t \in [t_1, t_p]. \quad (10)$$

In the prediction branch, the output distribution at time step t is conditioned on $\mathbf{z}^t$ and the hidden state h^{t-1} , together with the latent variable $\mathbf{z}^{t_p}$ extracted from the last available observation, expressed as:

$$\hat{\mathbf{y}}^t \sim \varphi_f^{\text{dec}}(\varphi_f^z(\mathbf{z}^t \oplus \mathbf{z}^{t_p}) \oplus h^{t-1}; \mathbf{W}_f^{\text{dec}}) \quad t \in [t_{p+1}, t_f], \quad (11)$$

where $\varphi^{\text{dec}}(\cdot)$ represents a decoder function with learnable weights $\mathbf{W}^{\text{dec}}$ that generates the parameters of the GMM distribution, and $\varphi^z(\cdot)$ serves to extract features from the latent variable $\mathbf{z}^t$. Within the prediction decoder, we concatenate $\mathbf{z}^t$

and z^{t_p} to construct joint feature representations, thereby ensuring that knowledge from the imputation branch is explicitly incorporated when generating the predictive distribution. During the testing phase, as our method generates multiple feasible trajectories, we select the trajectory corresponding to the maximum mixing weight at each step to decode and use it as input for the subsequent time step.

3) *Bidirectional Temporal Decay*: To model the temporal dynamics of missing observations, our earlier work [37] introduced the temporal lag δ_i^t , which measures the gap between the most recent available observation and the current time step t for agent i . The recurrence for computing δ_i^t is defined as:

$$\delta_i^t = \begin{cases} 1 + \delta_i^{t-1} & \text{if } t > 1 \text{ and } m_i^t = 0 \\ 1 & \text{if } t > 1 \text{ and } m_i^t = 1 \\ 0 & \text{if } t = 1 \end{cases} \quad (12)$$

By concatenating the lags δ_i^t across all agents, we obtain the temporal lag vector δ^t . The corresponding temporal decay vector Δ^t is then computed as:

$$\Delta^t = 1 / \exp(\max(0, \mathbf{W}_\delta \delta^t + \mathbf{b}_\delta)), \quad (13)$$

where $\mathbf{W}_\delta$ and $\mathbf{b}_\delta$ are the learnable weight matrix and bias.

In our proposed GMC-VRNN method, we extend this module to a bidirectional framework. Specifically, we reverse the mask of the observed trajectories along the time dimension and apply the same calculations as in (12) and (13) to generate the reversed temporal decay vector $\overleftarrow{\Delta}^t$. Different learnable weights and biases are used in (13) when projecting the temporal lags for the reversed direction.

The motivation for this design can be summarized as follows. In sequential modeling, when an observation remains missing for an extended period, its contribution to the representation should progressively diminish. The temporal lag δ^t quantifies the time gap from the most recent available observation to the current step, and thus exhibits an inverse relationship with the temporal decay factor. This relationship is modeled using a negative exponential function, ensuring a strictly monotonic decay that is naturally constrained to the interval [0,1]. The bidirectional strategy is introduced to fully explore the temporal missing patterns of the observed trajectory. This is particularly important for imputation tasks, where both past and future values play a critical role in estimating the missing values. Empirical results in Section IV confirm the effectiveness of this approach. Note that the decay vector is evaluated solely on the historical trajectories with missing observations, and is applied only in this context.

4) *Recurrence*: Modeling complex patterns of missing data cannot be achieved by temporal decay vectors alone. To enrich the information conveyed by the BTM mechanism, we introduce an operation that multiplies the hidden states element-wise with the sum of the forward and backward decay vectors during recurrent updates, formulated as:

$$h^{t-1'} = (\overrightarrow{\Delta}^t + \overleftarrow{\Delta}^t) \odot h^{t-1}. \quad (14)$$

This operation integrates temporally decayed features from both sequence directions. For the imputation branch, the RNN

is updated as follows:

$$h^t = \text{RNN}((\mathbf{F}_G^t \oplus \varphi_p^z(z^t)), h^{t-1'}) \\ t \in [t_1, t_p]. \quad (15)$$

For the prediction branch, the RNN is updated as:

$$h^t = \text{RNN}((\mathbf{F}_G^t \oplus \varphi_f^z(z^t \oplus z^{t_p})), h^{t-1}) \\ t \in [t_{p+1}, t_f]. \quad (16)$$

In the prediction branch, the latent variable z^t is concatenated with z^{t_p} to incorporate the final observed context into the hidden state updates. This concatenation ensures that the prediction branch incorporates both current and historical latent variable information for more accurate updates.

5) *Inference*: At each time step, the approximate posterior distribution of the latent variable is defined as:

$$z^t | x^t \sim \mathcal{N}(\mu^{\text{enc},t}, \sigma^{\text{enc},t,2}) \quad t \in [t_1, t_p] \\ z^t | y^t \sim \mathcal{N}(\mu^{\text{enc},t}, \sigma^{\text{enc},t,2}) \quad t \in [t_{p+1}, t_f]. \quad (17)$$

The posterior parameters are conditioned on both the graph embedding and the hidden state of the RNN, computed as:

$$[\mu^{\text{enc},t}, \sigma^{\text{enc},t,2}] = \varphi^{\text{enc}}((\mathbf{F}_G^t \oplus h^{t-1}); \mathbf{W}^{\text{enc}}), \quad (18)$$

where $\varphi^{\text{enc}}(\cdot)$ is the encoder network parameterized by $\mathbf{W}^{\text{enc}}$.

6) *Loss Function*: The model is trained by maximizing the Sequential ELBO introduced in (2), which is composed of two terms: $\mathcal{L}_{\text{imp}}$ for imputation and $\mathcal{L}_{\text{pre}}$ for prediction.

Given the multi-modal nature of the output distribution, we further minimize the negative log-likelihood of the ground-truth trajectory under the estimated Gaussian mixture. To associate the supervision signal with an appropriate mixture component, we adopt a hard-assignment strategy: the mixture component whose mean is closest to the ground truth (in Euclidean distance) is treated as the “correct” one. Formally, the imputation and prediction losses are expressed as:

$$\mathcal{L}_{\text{imp}} = - \sum_{k=1}^K \mathbb{1}(k = \hat{k}) \left[\log p_k + \sum_{t=t_1}^{t_p} \log(\mathbb{P}(x^t | \hat{\mu}^t, \hat{\sigma}^t, \hat{\rho}^t)) \right] \\ + \lambda_1 \text{KL}(\mathcal{N}(\mu^{\text{enc},t}, \sigma^{\text{enc},t,2}) || \mathcal{N}(\mu^{\text{pri},t}, \sigma^{\text{pri},t,2})), \quad (19) \\ \mathcal{L}_{\text{pre}} = - \sum_{k=1}^K \mathbb{1}(k = \hat{k}) \\ \left[\log p_k + \sum_{t=t_{p+1}}^{t_f} \log(\mathbb{P}(y^t | \hat{\mu}^t, \hat{\sigma}^t, \hat{\rho}^t)) \right] \\ + \lambda_2 \text{KL}(\mathcal{N}(\mu^{\text{enc},t}, \sigma^{\text{enc},t,2}) || \mathcal{N}(\mu^{\text{pri},t}, \sigma^{\text{pri},t,2})), \quad (20)$$

where $\mathbb{1}(\cdot)$ represents the indicator function, and $\hat{k}$ corresponds to the index of the mixture component selected via hard assignment. The total loss function is expressed as:

$$\mathcal{L} = \mathcal{L}_{\text{imp}} + \lambda_3 \mathcal{L}_{\text{pre}}, \quad (21)$$

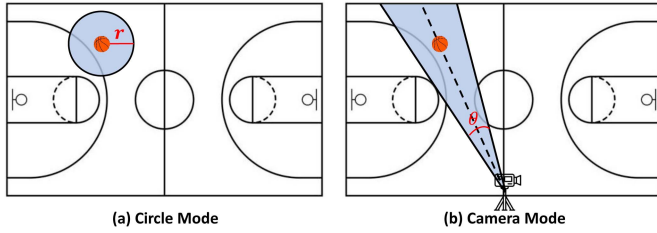


Fig. 4. Illustration of the two designed strategies, circle mode and camera mode, for simulating potential missing scenarios in sports trajectories.

where $\{\lambda_1, \lambda_2, \lambda_3\}$ are balancing factors, which in our experiments are set to 1. The objective is computed over all agents in each trajectory, and the network is optimized end-to-end for both imputation and prediction.

IV. EXPERIMENTS

This section begins by introducing the two sports datasets with incomplete observations, together with the strategies employed to simulate realistic missing scenarios. Implementation details of the proposed method are then described, followed by experimental results and ablation studies conducted on the two datasets across ten scenarios. Finally, visualization examples are provided to compare the proposed approach with representative baselines.

A. Experimental Settings

1) *Datasets*: Different from our previous work [37], we disregard the Vehicle-TIP dataset and focus on the sports game scenario due to the complicated and dense interactions among agents. For the Basketball-TIP and Football-TIP datasets, we improve dataset diversity by introducing more hyperparameters to collect the incomplete trajectories.

Basketball-TIP: The Basketball-TIP dataset is constructed from the NBA dataset [36] and contains 104,003 training sequences and 13,464 testing sequences. Each sequence consists of 11 trajectories (the ball, five offensive players, and five defensive players) spanning 50 frames over 8 seconds, with the court size being 94 feet by 50 feet. To emulate the realistic visibility changes of players during live games, two masking strategies are applied: circle mode and camera mode (Fig. 4). In circle mode, player visibility is determined by a ball-centered circle of radius r (in feet): agents inside remain visible, while those outside are masked. In camera mode, a virtual camera is placed at the sideline midpoint with a fixed field-of-view (FOV) angle θ (in degrees), whose bisector follows the ball, and the intersection of the FOV with the court defines the visible region. To enrich missing-pattern diversity, Basketball-TIP is expanded into ten scenarios using five different radii r and five FOV angles θ .

Football-TIP: The Football-TIP dataset is based on the NFL Big Data Bowl dataset¹, comprising 10,780 training sequences and 2,492 testing sequences. Each sequence contains 21 trajectories (the ball, ten offensive players, and ten defensive players) recorded over 50 frames within 10 seconds, with the field size

being 120 yards by 53.3 yards. The same circle-mode and camera-mode strategies are employed, where radii r (in yards) and FOV angles θ (in degrees) are varied across five levels each, producing ten scenarios for evaluation.

2) *Pre-Processing*: Sequences without ball trajectories are excluded. In addition, player trajectories located outside the court or field are removed, although such cases occasionally occur in real games. Finally, all sequence data are normalized to the range $[-1, 1]$ according to the corresponding court or field dimensions.

3) *Metrics*: The minimum Average Displacement Error (minADE) is adopted to assess the multi-modal performance of both imputation and prediction. For each sample, this metric identifies the candidate trajectory (among K generated) that has the smallest L_2 distance to the ground truth and then averages the error across all future time steps. In our experiments, I-minADE₂₀ is reported for imputation and P-minADE₂₀ for prediction.

4) *Baselines*: The evaluation includes a range of baselines. For both imputation and prediction, we consider Linear Fit, Vanilla LSTM [64], Vanilla VRNN [63], and INAM [57]. For imputation specifically, additional comparisons are made against Mean, Median, GMAT [36], NAOMI [35], and GC-VRNN [37].

5) *Implementation Details*: Across all scenarios in Basketball-TIP and Football-TIP, the observation length is set to 30 frames and the prediction length to 20 frames. The MS-GNN encoder is composed of three static topology GCLs and one dynamic learnable GCL, while the edge-conditioned GCL is only employed for observations. The mapping network $\varphi_{ec}(\cdot)$ is implemented as a Conv2D block consisting of two 1×1 Conv2D layers followed by a channel-level average pooling layer. The functions $\varphi_{pri}(\cdot)$, $\varphi_{enc}(\cdot)$, $\varphi_{dec}(\cdot)$, and $\varphi_z(\cdot)$ are realized as MLPs. The node feature dimension D in MS-GNN is fixed to 32. The RNN hidden size is 256, and the latent variable dimension is 64. All experiments are conducted in PyTorch [65] on an Nvidia A100 GPU. The model is trained for 100 epochs with a batch size of 64, optimized using Adam [66] with an initial learning rate of 1×10^{-4} and a decay factor of 0.9 applied every 20 epochs.

B. Main Results

The trajectory imputation and prediction results on the Basketball-TIP and Football-TIP datasets are reported in Tables I and II, respectively. Our proposed GMC-VRNN achieves state-of-the-art performance consistently across all ten scenarios in both imputation (I- L_2) and prediction (P- L_2), evidencing its robustness in handling heterogeneous missing data patterns. Compared to traditional baselines such as Linear Fit, GMAT, and NAOMI, our method shows significant improvements, particularly in challenging scenarios with higher missing rates, such as smaller radii (r) in circle mode or smaller field-of-view angles (θ) in camera mode. For example, on the Basketball-TIP dataset, GMC-VRNN achieves a substantial reduction in errors over GC-VRNN, with improvements of 3.55 (I- L_2) and 1.35 (P- L_2) in the hardest scenario $\theta = 10^\circ$. Similarly, on the Football-TIP dataset, GMC-VRNN outperforms GC-VRNN by 3.09 (I- L_2) and 4.49 (P- L_2) under the most

¹<https://github.com/nfl-football-ops/Big-Data-Bowl>

TABLE I
QUANTITATIVE EVALUATION ON THE BASKETBALL-TIP DATASET (IN FEET) UNDER TEN SCENARIOS, CONSTRUCTED BY VARYING RADIUS r AND ANGLE θ IN THE TWO MASKING STRATEGIES

Method	$r = 3$ ft.		$r = 5$ ft.		$r = 7$ ft.		$r = 9$ ft.		$r = 11$ ft.		$\theta = 10^\circ$		$\theta = 15^\circ$		$\theta = 20^\circ$		$\theta = 25^\circ$		$\theta = 30^\circ$	
	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2
Mean	9.07	—	9.53	—	9.51	—	9.71	—	9.80	—	8.83	—	8.78	—	8.64	—	8.49	—	8.47	—
Median	9.32	—	9.82	—	9.81	—	9.97	—	10.12	—	9.16	—	9.11	—	8.96	—	8.92	—	8.75	—
GMAT [36]	7.36	—	6.89	—	6.73	—	6.54	—	6.48	—	6.42	—	6.21	—	5.99	—	5.87	—	6.01	—
NAOMI [35]	7.68	—	7.08	—	7.04	—	6.98	—	6.88	—	6.33	—	6.24	—	6.11	—	6.12	—	5.91	—
Linear Fit	14.90	21.14	14.06	20.36	13.58	18.94	13.21	17.86	12.74	17.22	12.78	21.01	11.91	18.72	11.47	16.38	11.44	15.88	11.26	14.40
Vanilla LSTM [64]	7.33	20.07	6.73	14.91	6.51	10.07	6.42	9.87	6.40	9.74	6.28	9.34	6.24	8.99	6.01	7.52	5.73	6.22	5.67	6.10
Vanilla VRNN [63]	7.43	12.26	6.90	11.38	6.68	10.07	6.48	9.42	6.44	9.27	6.38	8.49	6.10	7.52	6.09	7.47	6.02	7.44	5.92	7.36
INAM [57]	7.35	8.93	6.93	8.24	6.80	7.68	6.78	7.60	6.62	7.44	6.50	7.32	6.21	7.14	6.13	7.10	6.12	7.08	5.92	6.96
GC-VRNN [37]	7.03	7.50	6.41	6.80	6.24	5.93	6.09	5.47	6.04	5.28	5.86	6.29	5.68	4.97	5.56	4.74	5.44	4.52	5.39	4.28
GMC-VRNN (Ours)	2.81	3.84	2.51	3.60	2.45	3.28	2.49	3.31	2.50	3.45	2.31	4.94	2.24	3.34	2.23	3.47	2.23	3.48	2.26	3.52

TABLE II
QUANTITATIVE EVALUATION ON THE FOOTBALL-TIP DATASET (IN YARDS) UNDER TEN SCENARIOS, CONSTRUCTED BY VARYING RADIUS r AND ANGLE θ IN THE TWO MASKING STRATEGIES

Method	$r = 2$ yd.		$r = 3$ yd.		$r = 4$ yd.		$r = 5$ yd.		$r = 6$ yd.		$\theta = 2^\circ$		$\theta = 4^\circ$		$\theta = 6^\circ$		$\theta = 8^\circ$		$\theta = 10^\circ$	
	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2
Mean	8.94	—	9.10	—	9.28	—	9.44	—	9.69	—	8.74	—	8.86	—	9.12	—	9.27	—	9.42	—
Median	8.99	—	9.12	—	9.39	—	9.45	—	9.86	—	8.80	—	8.93	—	9.23	—	9.39	—	9.44	—
GMAT [36]	4.63	—	5.22	—	5.37	—	5.89	—	6.44	—	7.37	—	7.22	—	6.98	—	6.92	—	6.84	—
NAOMI [35]	4.48	—	4.67	—	4.95	—	5.56	—	5.83	—	7.21	—	7.18	—	6.82	—	6.70	—	6.67	—
Linear Fit	7.18	7.58	7.14	7.44	7.01	6.97	7.06	7.22	7.08	9.88	8.48	9.77	7.45	9.02	7.17	8.40	7.12	8.04	6.85	7.94
Vanilla LSTM [64]	5.26	6.98	5.41	6.64	5.96	5.13	6.27	6.25	6.47	6.83	7.33	10.21	7.52	9.47	7.85	7.90	7.75	7.88	7.40	7.24
Vanilla VRNN [63]	4.61	5.63	4.88	5.69	5.31	5.48	5.87	5.52	6.29	5.94	6.70	8.27	6.41	7.23	6.38	6.98	6.21	6.72	6.24	6.68
INAM [57]	4.32	6.01	4.65	5.76	4.94	5.52	5.49	5.98	6.11	6.34	7.19	8.31	7.14	8.24	6.85	7.33	6.62	7.26	6.57	7.09
GC-VRNN [37]	3.95	4.50	4.24	4.59	4.68	4.42	4.87	4.48	5.19	4.66	5.54	7.58	5.52	7.47	5.37	5.88	5.42	5.71	5.22	5.19
GMC-VRNN (Ours)	2.31	2.66	2.30	2.73	2.36	2.61	2.54	2.63	2.78	2.66	2.45	3.09	2.47	3.16	2.56	3.03	2.64	3.02	2.76	3.06

challenging conditions $\theta = 2^\circ$. This highlights the ability of GMC-VRNN to capture spatio-temporal dependencies more effectively than previous methods, particularly in scenarios with extremely missing observations.

Furthermore, the improvements over GC-VRNN across all scenarios validate the effectiveness of our enhanced module designs, including the bidirectional temporal decay module and the Gaussian Mixture Model (GMM)-based trajectory representation, which better handle multi-modal trajectory data and account for missing patterns. Simple baselines such as Mean and Median fail to consider temporal and spatial relationships, resulting in poor performance. While methods such as NAOMI and GMAT perform better, they still fall short compared to GMC-VRNN, under conditions with high missing rates. Additionally, GMC-VRNN demonstrates its generalizability by delivering consistent performance across both datasets, despite differences in spatial scale (feet vs. yards) and the number of agents (11 vs. 21 trajectories). These results confirm that GMC-VRNN is a robust and effective solution in real-world settings.

C. Ablation Study

We conduct ablation experiments to quantify the contribution of each component within our proposed framework.

1) *Three GCLs*: Table III reports the results of the component analysis for the three GCL variants: Static Topology, Dynamic Learnable, and Edge-Conditioned. Each variant provides measurable performance gains, confirming the effectiveness of its respective design. Among these, the Edge Conditioned (EC) GCL yields the most significant improvement in accuracy. For example, the inclusion of the EC GCL reduces I- L_2 and P- L_2 errors consistently across all scenarios, with noticeable improvements in both $r = 3$ ft. and $\theta = 10^\circ$ scenarios. This highlights

TABLE III
ABLATION STUDY ON THE MS-GNN, EVALUATING THE CONTRIBUTIONS OF THE THREE GCL VARIANTS: STATIC TOPOLOGY (ST), DYNAMIC LEARNABLE (DL), AND EDGE-CONDITIONED (EC)

ID		GCL			$r = 3$ ft.		$r = 5$ ft.		$\theta = 10^\circ$		$\theta = 15^\circ$	
		ST	DL	EC	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2
1	✓	✓	✓	✓	3.14	4.46	3.26	5.01	3.20	4.30	3.78	6.28
2	✓	✓	✓	✓	2.97	4.24	3.18	4.98	3.04	4.12	3.55	5.97
3	✓	✓	✓	✓	2.95	4.12	3.10	4.67	3.01	3.97	3.48	5.74
Ours	✓	✓	✓	✓	2.81	3.84	2.51	3.60	2.31	4.94	2.24	3.34

TABLE IV
EVALUATION OF THE BIDIRECTIONAL TEMPORAL DECAY (BTD) MODULE AND THE INTEGRATION MECHANISM LINKING THE IMPUTATION AND PREDICTION BRANCHES

Variants	$r = 3$ ft.		$r = 5$ ft.		$\theta = 10^\circ$		$\theta = 15^\circ$	
	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2	I- L_2	P- L_2
w/o BTD	3.02	4.18	2.64	3.85	3.78	3.71	2.59	5.29
w/ IMP	3.18	23.98	2.67	24.49	2.83	27.07	2.81	26.97
w/ PRE	23.16	3.99	23.10	4.10	25.17	3.78	25.79	5.38
w/ CON	2.98	13.45	2.56	12.74	2.71	15.02	2.66	14.88
Ours	2.81	3.84	2.51	3.60	2.31	4.94	2.24	3.34

the EC GCL's ability to capture spatial missing patterns from incomplete observations, which is critical for improving the model's spatial feature extraction capabilities. The results also indicate that combining all three GCLs provides the highest performance, as the complementary strengths of each layer enhance the model's overall ability to encode spatial features.

2) *Bidirectional Temporal Decay*: Table IV shows the effect of the Bidirectional Temporal Decay (BTD) module, which extends the unidirectional variant proposed in our earlier work to more effectively capture temporal missing patterns. Excluding the BTD module ("w/o BTD") results in substantial performance reductions for both imputation and prediction, especially in settings characterized by high missing rates. For example, in the

TABLE V

ABLATION STUDY ON KEY HYPERPARAMETERS OF GMC-VRNN ON THE BASKETBALL-TIP DATASET. RESULTS ARE REPORTED UNDER TWO SETTINGS: DISTANCE-BASED ($r = 3$ Ft) AND ANGLE-BASED ($\theta = 10^\circ$).

Setting	$r = 3$ ft.		$\theta = 10^\circ$	
	I- L_2	P- L_2	I- L_2	P- L_2
<i>Number of GCN layers</i>				
2	3.12	4.25	2.54	5.31
3	2.81	3.84	2.31	4.94
4	3.05	4.12	2.47	5.20
<i>Latent variable dimension</i>				
64	2.81	3.84	2.31	4.94
128	2.95	4.05	2.44	5.08
256	3.10	4.22	2.53	5.25
<i>RNN hidden size</i>				
128	2.93	4.01	2.45	5.09
256	2.81	3.84	2.31	4.94

$\theta = 15^\circ$ scenario, the absence of BTD increases the I- L_2 error from 2.31 to 2.59 and the P- L_2 error from 4.94 to 5.29. These results confirm that modeling temporal decay in both forward and backward directions enhances the model's ability to process incomplete temporal information, which is essential for both tasks.

3) *Connection*: To investigate the benefit of jointly considering imputation and prediction, we examine three settings: “w/ IMP” means imputation only, “w/ PRE” means prediction only, and “wo/ CON” means decoupling imputation and prediction by using separate RNNs. Table IV shows that jointly handling these tasks in a unified framework leads to substantial performance gains, particularly for prediction. For example, when comparing “w/ PRE” to the unified framework, the P- L_2 error in the $r = 5$ ft. scenario increases from 3.60 to 4.10, demonstrating that imputation provides valuable temporal context for prediction. Similarly, decoupling the tasks (“wo/ CON”) leads to degraded performance in both I- L_2 and P- L_2 , further validating the importance of the shared parameter space and mutual information exchange between the two tasks.

Overall, these ablation studies highlight the effectiveness of each proposed component and confirm the necessity of considering both imputation and prediction tasks simultaneously. Combining the three GCLs with the BTD module within the joint framework yields notable improvements in handling incomplete observations and in generating precise predictions.

4) *Hyperparameters*: Table V reports ablations on the number of GCN layers, latent variable dimension, and RNN hidden size. We observe that three GCN layers consistently outperform both two and four: two layers provide an insufficient receptive field, whereas four layers tend to cause oversmoothing without further gains. Regarding the latent variable dimension, a size of 64 achieves the most robust generalization across settings. Increasing the dimension to 128 or 256 does not yield consistent improvements but increases computation and shows a tendency to overfit. For the RNN hidden size, 256 attains the best accuracy on all metrics, while reducing it to 128 results in slightly worse performance, albeit with a modest runtime reduction. Overall, these comparisons justify our hyperparameter choices

TABLE VI

AVERAGE INFERENCE TIME PER SEQUENCE (MS) ON THE TEST SETS

Method	Basketball-TIP $N=11$	Football-TIP $N=21$
Vanilla LSTM [64]	7.8	13.9
Vanilla VRNN [63]	9.5	16.8
INAM [57]	11.2	18.5
GC-VRNN [37]	10.8	19.2
GMC-VRNN (Ours)	12.4	21.7

and demonstrate that they strike a balance between accuracy and scalability.

D. Complexity and Runtime Analysis

We analyze both the theoretical complexity and the practical runtime efficiency of GMC-VRNN.

1) *MS-GNN*: At each time step with N agents and hidden dimension D , each graph convolutional layer (static, dynamic, and edge-conditioned) has complexity $O(N^2D)$ due to adjacency-based aggregation. Over T time steps, the overall complexity is $O(TN^2D)$.

2) *GMM*: The Gaussian Mixture Model decoder generates the parameters of K mixture components for each prediction step. With prediction horizon T_p and spatial dimension $d = 2$, the complexity is $O(T_p K d)$, which is linear in T_p and K .

3) *BTB*: The BTB module computes temporal lags and decay vectors in both forward and backward directions, requiring $O(TH)$ operations with hidden size H . This cost is minor compared with MS-GNN and the decoder.

4) *Overall*: The dominant cost arises from MS-GNN, which scales quadratically with the number of agents N . In sports scenarios with modest N (e.g., $N=11$ for basketball and $N=21$ for football), the framework remains computationally efficient and scalable to real-world applications.

5) *Runtime Analysis*: We report the average inference time on the test sets. All methods are implemented in PyTorch and executed on a single NVIDIA A100 GPU with a batch size of 64. As shown in Table VI, GMC-VRNN achieves 12.4 ms per sequence on Basketball-TIP ($N = 11$) and 21.7 ms on Football-TIP ($N = 21$), which is only slightly higher than GC-VRNN (10.8 ms and 19.2 ms, respectively). The additional overhead is negligible, as the inference speed remains far below the frame interval of broadcast sports videos (25–30 FPS), thereby confirming the suitability of GMC-VRNN for real-time applications. We do not report inference times for methods such as GMAT and NAOMI, since they are designed for imputation only and cannot be directly applied to the prediction setting. Likewise, baselines such as Linear Fit, Mean, and Median are excluded because they are non-neural approaches with negligible runtime but substantially worse accuracy, making them unsuitable for practical deployment.

E. Qualitative Results

Fig. 5 presents qualitative examples of trajectory imputation on the Basketball-TIP dataset ($\theta = 10^\circ$). Overall, our GMC-VRNN produces imputed trajectories that are substantially more

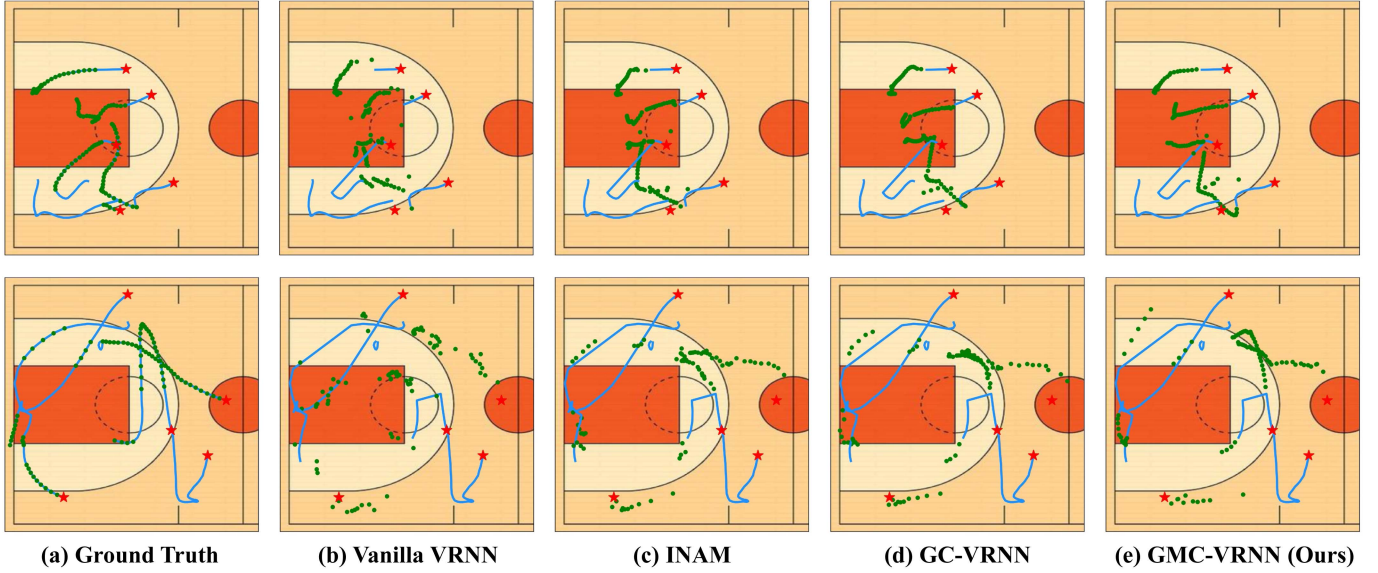


Fig. 5. Qualitative results of trajectory imputation on Basketball-TIP ($\theta = 10^\circ$). Starting points are shown as red stars, visible observations as blue lines, and missing entries as green points. For clarity of presentation, only five defenders are included.

accurate than those generated by baseline methods, underscoring the effectiveness of our proposed approach. In the first-row example, the trajectories predicted by GMC-VRNN align more closely with the ground truth compared with Vanilla VRNN, INAM, and GC-VRNN, successfully capturing the underlying motion pattern despite incomplete observations. A similar pattern is observed in the second-row example, where GMC-VRNN again produces more realistic and accurate trajectory reconstructions, outperforming all competing baselines.

Despite the improvements observed, the second-row example shows that the bottom agent's trajectory is not accurately reconstructed by any method, reflecting the persistent difficulty of handling severely incomplete observations. This limitation can be attributed to the prolonged and extensive absence of observations for this trajectory, which makes both imputation and prediction particularly challenging. While our method still offers an improvement over other baselines, the performance gap suggests that leveraging limited observations for better trajectory reconstruction remains an open problem. This limitation highlights an important direction for future research: developing techniques to better handle scenarios with long-duration missing data. Incorporating additional temporal or spatial priors, leveraging external knowledge, or employing advanced generative models may help address these challenges and further improve performance in such extreme cases.

V. CONCLUSION

This work presents GMC-VRNN, a unified framework that extends the earlier GC-VRNN model to jointly address trajectory prediction and imputation. GMC-VRNN incorporates a Multi-Space Graph Neural Network (MS-GNN) and a Gaussian Mixture VRNN with a Bidirectional Temporal Decay (BTD) module, enabling the extraction of comprehensive spatio-temporal features from incomplete observations. In addition, two sports datasets were expanded with diverse

scenarios to facilitate the evaluation of the joint task. Extensive experiments show that GMC-VRNN consistently outperforms strong baselines, demonstrating its robustness and effectiveness in handling complex real-world settings.

REFERENCES

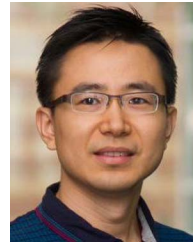
- [1] J. Wang, T. Ye, Z. Gu, and J. Chen, "LTP: Lane-based trajectory prediction for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 17134–17142.
- [2] C. Hazard et al., "Importance is in your attention: Agent importance prediction for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 2532–2535.
- [3] S. Hu, L. Chen, P. Wu, H. Li, J. Yan, and D. Tao, "ST-P3: End-to-end vision-based autonomous driving via spatial-temporal feature learning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 533–549.
- [4] Y. Xu et al., "VLM-AD: End-to-end autonomous driving through vision-language model supervision," 2024, *arXiv:2412.14446*.
- [5] S. Suwajanakorn, S. M. Seitz, and I. Kemelmacher-Shlizerman, "Synthesizing obama: Learning lip sync from audio," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–13, 2017.
- [6] S. Taylor et al., "A deep learning approach for generalized speech animation," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–11, 2017.
- [7] H. Girase et al., "Loki: Long term and key intentions for trajectory prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9803–9812.
- [8] H. Ma, Y. Sun, J. Li, M. Tomizuka, and C. Choi, "Continual multi-agent interaction behavior prediction with conditional generative memory," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8410–8417, 2021.
- [9] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Phys. Rev. E*, vol. 51, no. 5, 1995, Art. no. 4282.
- [10] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social LSTM: Human trajectory prediction in crowded spaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 961–971.
- [11] J. Liang, L. Jiang, J. C. Niebles, A. G. Hauptmann, and L. Fei-Fei, "Peeking into the future: Predicting future person activities and locations in videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 5725–5734.
- [12] P. Zhang, W. Ouyang, P. Zhang, J. Xue, and N. Zheng, "SR-LSTM: State refinement for LSTM towards pedestrian trajectory prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12085–12094.
- [13] Y. Hu, S. Chen, Y. Zhang, and X. Gu, "Collaborative motion prediction via neural motion message passing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 6319–6328.

- [14] Y. Xu, J. Yang, and S. Du, "CF-LSTM: Cascaded feature-based long short-term networks for predicting pedestrian trajectory," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12541–12548.
- [15] Y. Zhu et al., "Simultaneous past and current social interaction-aware trajectory prediction for multiple intelligent agents in dynamic scenes," *ACM Trans. Intell. Syst. Technol.*, vol. 13, pp. 1–16, 2021.
- [16] T. Fernando, S. Denman, S. Sridharan, and C. Fookes, "GD-GAN: Generative adversarial networks for trajectory prediction and group detection in crowds," in *Proc. Asian Conf. Comput. Vis.*, 2018, pp. 314–330.
- [17] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, "Social GAN: Socially acceptable trajectories with generative adversarial networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2255–2264.
- [18] J. Li, H. Ma, and M. Tomizuka, "Conditional generative neural system for probabilistic trajectory prediction," 2019, *arXiv: 1905.01631*.
- [19] A. Sadeghian, V. Kosaraju, A. Sadeghian, N. Hirose, H. Rezatofighi, and S. Savarese, "SoPhie: An attentive GAN for predicting paths compliant to social and physical constraints," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1349–1358.
- [20] P. Dendorfer, S. Elflein, and L. Leal-Taixé, "MG-GAN: A multi-generator model preventing out-of-distribution samples in pedestrian trajectory prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 13158–13167.
- [21] K. Mangalam et al., "It is not the journey but the destination: Endpoint conditioned trajectory prediction," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 759–776.
- [22] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron: Dynamically-feasible trajectory forecasting with heterogeneous data," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 683–700.
- [23] Y. Xu, D. Ren, M. Li, Y. Chen, M. Fan, and H. Xia, "Tra2Tra: Trajectory-to-trajectory prediction with a global social spatial-temporal attentive neural network," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1574–1581, Apr. 2021.
- [24] G. Chen, J. Li, J. Lu, and J. Zhou, "Human trajectory prediction via counterfactual analysis," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9824–9833.
- [25] J. Yoon, J. Jordon, and M. Schaar, "Gain: Missing data imputation using generative adversarial nets," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5689–5698.
- [26] W. Fedus, I. Goodfellow, and A. M. Dai, "MaskGAN: Better text generation via filling in the _," in *Proc. Int. Conf. Learn. Representation*, 2018.
- [27] Y. Luo, X. Cai, Y. Zhang, J. Xu, and X. Yuan, "Multivariate time series imputation with generative adversarial networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1603–1614.
- [28] J. Yi, J. Lee, K. J. Kim, S. J. Hwang, and E. Yang, "Why not to use zero imputation? Correcting sparsity bias in training neural networks," in *Proc. Int. Conf. Learn. Representation*, 2020, pp. 1–27.
- [29] S. N. Shukla and B. Marlin, "Multi-time attention networks for irregularly sampled time series," in *Proc. Int. Conf. Learn. Representation*, 2021, pp. 1–15.
- [30] A. Cini, I. Marisca, and C. Alippi, "Multivariate time series imputation by graph neural networks," in *Proc. Int. Conf. Learn. representation*, 2022, pp. 1–20.
- [31] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Appl. Soft Comput.*, vol. 90, 2020, Art. no. 106181.
- [32] I. Silva, G. Moody, D. J. Scott, L. A. Celi, and R. G. Mark, "Predicting in-hospital mortality of ICU patients: The physioNet/computing in cardiology challenge 2012," in *Proc. Comput. Cardiol.*, 2012, pp. 245–248.
- [33] A. E. Johnson et al., "MIMIC-III, a freely accessible critical care database," *Sci. Data*, vol. 3, no. 1, pp. 1–9, 2016.
- [34] Y. Zheng, L. Capra, O. Wolfson, and H. Yang, "Urban computing: Concepts, methodologies, and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 5, no. 3, pp. 1–55, 2014.
- [35] Y. Liu, R. Yu, S. Zheng, E. Zhan, and Y. Yue, "Naomi: Non-autoregressive multiresolution sequence imputation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 11238–11248.
- [36] E. Zhan, S. Zheng, Y. Yue, L. Sha, and P. Lucey, "Generating multi-agent trajectories using programmatic weak supervision," in *Proc. Int. Conf. Learn. Representation*, 2019, pp. 1–14.
- [37] Y. Xu, A. Bazarjani, H.-G. Chi, C. Choi, and Y. Fu, "Uncovering the missing pattern: Unified framework towards trajectory imputation and prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 9632–9643.
- [38] A. Vemula, K. Muelling, and J. Oh, "Social attention: Modeling attention in human crowds," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 1–7.
- [39] J. Amirian, J.-B. Hayet, and J. Pettré, "Social ways: Learning multimodal distributions of pedestrian trajectories with GANs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019, pp. 2964–2972.
- [40] V. Kosaraju, A. Sadeghian, R. Martín-Martín, I. Reid, H. Rezatofighi, and S. Savarese, "Social-BiGAT: Multimodal trajectory forecasting using bicycle-gan and graph attention networks," in *Adv. Neural Inf. Process. Syst.*, 2019, pp. 137–146.
- [41] C. Xu, W. Mao, W. Zhang, and S. Chen, "Remember intentions: Retrospective-memory-based trajectory prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 6488–6497.
- [42] B. Ivanovic and M. Pavone, "The trajectron: Probabilistic multi-agent trajectory modeling with dynamic spatiotemporal graphs," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 2375–2384.
- [43] P. Xu, J.-B. Hayet, and I. Karamouzas, "SocialVAE: Human trajectory prediction using timewise latents," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 511–528.
- [44] T. Gu et al., "Stochastic trajectory prediction via motion indeterminacy diffusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 17113–17122.
- [45] W. Mao, C. Xu, Q. Zhu, S. Chen, and Y. Wang, "Leapfrog diffusion model for stochastic trajectory prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5517–5526.
- [46] C. Jiang et al., "Motiondiffuser: Controllable multi-agent motion prediction using diffusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 9644–9653.
- [47] E. Acuna and C. Rodriguez, "The treatment of missing values and its effect on classifier accuracy," in *Proc. Int. Federation Classification Societies, Classif., Clust., Data Min. Appl.*, 2004, pp. 639–647.
- [48] C. F. Ansley and R. Kohn, "On the estimation of arima models with missing values," in *Proc. Time Ser. Anal. Irregularly Observed Data*, 1984, pp. 9–37.
- [49] O. Troyanskaya et al., "Missing value estimation methods for dna microarrays," *Bioinformatics*, vol. 17, no. 6, pp. 520–525, 2001.
- [50] L. Beretta and A. Santaniello, "Nearest neighbor imputation algorithms: A critical evaluation," *BMC Med. Inf. Decis. Mak.*, vol. 16, no. 3, pp. 197–208, 2016.
- [51] Z. Ghahramani and M. Jordan, "Supervised learning from incomplete data via an EM approach," in *Proc. Adv. Neural Inf. Process. Syst.*, 1994, pp. 120–127.
- [52] F. V. Nelwamondo, S. Mohamed, and T. Marwala, "Missing data: A comparison of neural network and expectation maximization techniques," *Curr. Sci.*, vol. 93, no. 11, pp. 1514–1521, 2007.
- [53] Z. C. Lipton, D. Kale, and R. Wetzel, "Directly modeling missing data in sequences with RNNs: Improved classification of clinical time series," in *Proc. Mach. Learn. Healthc. Conf.*, 2016, pp. 253–270.
- [54] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Sci. Rep.*, vol. 8, no. 1, pp. 1–12, 2018.
- [55] J. Yoon, W. R. Zame, and M. van der Schaar, "Estimating missing data in temporal data streams using multi-directional recurrent neural networks," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 5, pp. 1477–1490, May 2019.
- [56] W. Cao, D. Wang, J. Li, H. Zhou, Y. Li, and L. Li, "Brits: Bidirectional recurrent imputation for time series," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6776–6786.
- [57] M. Qi, J. Qin, Y. Wu, and Y. Yang, "Imitative non-autoregressive modeling for trajectory forecasting and imputation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12736–12745.
- [58] V. Fortuin, D. Baranchuk, G. Raetsch, and S. Mandt, "GP-VAE: Deep probabilistic time series imputation," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2020, pp. 1651–1661.
- [59] X. Miao, Y. Wu, J. Wang, Y. Gao, X. Mao, and J. Yin, "Generative semi-supervised learning for multivariate time series imputation," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 8983–8991.
- [60] Y. Tashiro, J. Song, Y. Song, and S. Ermon, "CSDI: Conditional score-based diffusion models for probabilistic time series imputation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 24804–24816.
- [61] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. representation*, 2017, pp. 1–14.

- [62] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12026–12035.
- [63] J. Chung, K. Kastner, L. Dinh, K. Goel, A. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.
- [64] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [65] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [66] P. K. Diederik and B. Jimmy, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. representation*, 2015, pp. 1–15.



Yi Xu (Graduate Student Member, IEEE) received the BS and MS degrees from Xi'an Jiaotong University, in 2017 and 2020, respectively. He is currently working toward the PhD degree with Northeastern University, Boston, USA. His current research interests are machine learning, computer vision, autonomous driving, and vision-and-language models.



Yun Fu (Fellow, IEEE) received the BEng degree in information engineering, the MEng degree in pattern recognition and intelligence systems from Xi'an Jiaotong University, China, and the MS degree in statistics and PhD degree in electrical and computer engineering from the University of Illinois at Urbana-Champaign. He has been an interdisciplinary faculty member with the College of Engineering and the College of Computer and Information Science, Northeastern University, since 2012. His research interests are machine learning, computational intelligence, Big Data mining, computer vision, pattern recognition, and cyber-physical systems. He has extensive publications in leading journals, books/book chapters, and international conferences/workshops. He serves as an associate editor, the chair, a PC member, and a reviewer of many top journals and international conferences/workshops. He received seven prestigious young investigator awards from NAE, ONR, ARO, IEEE, INNS, UIUC, and Grainger Foundation; eleven best paper awards from IEEE, ACM, IAPR, SPIE, and SIAM; and many major industrial research awards from Google, Samsung, Amazon, Konica Minolta, JP Morgan, Zebra, Adobe, and Mathworks. He is currently an associate editor of *IEEE Transactions on Pattern Analysis and Machine Intelligence*. He is member of Academia Europaea (MAE), member of European Academy of Sciences and Arts (EASA), National Academy of Inventors (NAI) fellow, AAAI fellow, AAAS fellow, AIMBE fellow, IAPR fellow, OSA fellow, SPIE fellow, AAIA fellow, and ACM distinguished scientist.